Lab 2: Fine-tune vs prompt

Does fine-tuning beat your best prompt?

Aarne Talman

5 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	**3 / 5 Nov**	**Post-training**	**Lab 2: tune vs prompt**	**Aarne**
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

The question

One question

Lab 2

For *your* task, does fine-tuning beat your best prompt?

You compare two ways of getting a model to do your task:

- **prompting**: give the model instructions and a few examples, without changing it
- **fine-tuning**: train the model on your labelled examples

Both are scored on the same test examples, which neither has seen.

Everything you need is in `Week-2/Lab-2/README.md`, step by step.

What you take away

One results table, one loss plot and a short discussion for the final report.

You can reuse the fine-tuned model in week 3.

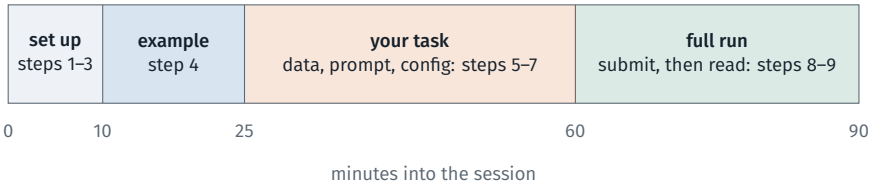
Material for the final report

Setting	Test score	What it is
Zero-shot		instructions only, no examples
Best prompt		the best of four prompts, chosen on the dev set
Fine-tuned (LoRA)		the model after training on your data
General knowledge	before, after	50 questions, before and after training

Plus one loss plot, showing the training loss and the dev loss.

Plus one paragraph: was fine-tuning worth the extra time and GPU use, compared with the points it gained?

Today, minute by minute



This briefing stops at minute 10. If the smoke test has not passed by minute 15, put your hand up.

Steps 1–9 refer to `Week-2/Lab-2/README.md`.

The full experiment is **one GPU job**: it scores the prompts, trains the model and runs the general-knowledge check, one after the other. It can take an hour or more, so submit it by minute 60, even with a first version of your prompt. You can improve the prompt and run it again later.

Baselines

Four prompting baselines

	Zero-shot instructions only	**Few-shot** instructions + 5 examples
Base model		
Instruct model		

A **base model** has only learned to continue text. An **instruct model** has also been trained to follow instructions.

The script scores all four on the **dev set**. The best one becomes *the* baseline, and only that one is compared with fine-tuning on the test set.

Why four?

Beating a weak prompt says little. Fine-tuning has to beat the best prompt you have, not the first one you tried.

A fair comparison

The same for every run

- no randomness: the model always writes its most likely answer
- the same prompt format
- the same scorer, which reads `Label: <label>` from the output
- a fixed random seed, recorded in the results

These are already set in the configuration file. Leave them as they are.

Three splits of your data

- **train**: examples the model learns from (1,000–5,000)
- **dev**: for choosing the prompt and the training settings
- **test**: only for the final scores

The few-shot examples come from train, never from test.

In the report, give the size of each split and the metric (here, accuracy).

Three ways to get a misleading baseline

1. The scorer misreads the answer. The model wrote "I think this review is positive", but the scorer looked only for `Label: positive` and found nothing. *Check:* read ten outputs with `show_examples.py`.

2. A chat format on a base model. Base models have not learned the chat format of instruct models. The scripts give base models plain text, which is the right choice.

3. The model has seen the test data. Public datasets are often part of the text a model was trained on, so a high zero-shot score may partly be memory. *Check:* you cannot rule this out, so mention it in the report. Week 4 returns to this.

Read the outputs before you trust a score

A score of exactly 0 or 100 usually means the scorer is wrong, not the model.

Read ten raw outputs next to the predicted and correct labels: `show_examples.py --run <name>`.

Problems 1 and 2 make the baseline look worse than it is; problem 3 can make it look better. Either way, the comparison with fine-tuning becomes unfair.

Your task

Your data and your prompt

```
# data/my-task/train.jsonl: one example per line
{"id": "train-00001", "text": "I left before the end.", "label": "negative"}

# lab2lib/prompting.py: the two settings you change
TASK_FIELDS = ("text",)
TASK_SYSTEM = ("Is the review positive or negative? "
               "Use exactly one of: positive, negative. "
               "End with 'Label: <label>'.")
```

- `make_splits.py` turns your data file into train, dev and test files; `check_data.py` checks them.
- **`TASK_FIELDS`** names the input fields; **`TASK_SYSTEM`** is the instruction. Keep the last sentence, because the scorer looks for `Label: <label>`.
- Fine-tuning teaches the model to answer `Label: <label>`. Only the answer counts in training, not the prompt.
- Read your prompts in a test run without a GPU before you submit the real job (README, steps 5–7).

Training settings: three you may change

Setting	Start	What it controls
learning rate	0.0002	how large each training step is
epochs	2	how many times the model sees the training data
LoRA rank	16	the size of the trained add-on (the adapter)

If you change the rank, set `lora_alpha` to twice the rank. Leave all other settings in the config as they are.

Source: The starting values are common defaults, not tuned for your task.

How to tune

1. Change *one* setting at a time.
2. Give each run its own name and output folder.
3. Choose on the **dev loss**, not on the test score.

Keep a small table of every run: setting, value, training loss, dev loss, test score. The table is part of your result.

When the training loss falls but nothing else improves

Lower training loss is not a result

If the training loss falls while the dev loss stays flat or rises, the model is **memorising** the training examples instead of learning the task.

With small training sets this can happen within one epoch.

The **loss** measures how wrong the model's answers are on the training data or on the dev data. Lower is better.

What to do

- look at the loss plot: it shows **both curves**
- note the point where the dev loss starts to rise
- try fewer epochs first, then a lower learning rate, then a smaller rank

The general-knowledge check

50 general-knowledge questions, unrelated to your task.

The script asks them **twice**: before training (the instruct model) and after training (the fine-tuned model). It uses the same questions and the same scorer both times.

What matters is the **change**, not the score itself.

A few points lower is normal. Twenty points lower means training has damaged the model: try fewer epochs or a lower learning rate.

Why check this?

Training a model on one narrow task can make it worse at other things. This is called **catastrophic forgetting**, and your task score does not show it.

50 questions is a small sample

Each question is worth 2 points, and a difference of up to about 7 points can be chance. Report both numbers and do not read much into a small change.

Before you leave

The report table and files to keep

Setting	Score	Split
Base, zero-shot		dev, test
Base, 5 examples		dev, test
Instruct, zero-shot		dev, test
Instruct, 5 examples		dev, test
Fine-tuned (LoRA)		test
General knowledge, before		50 Q
General knowledge, after		50 Q

`summarize_results.py` prints this table. Add a caption with the dataset, split sizes, metric and model names.

Keep for the final report

- your config file and `prompting.py`
- `results.json`, with every number above
- `predictions/`, the raw model outputs
- `loss.png`, the loss plot
- the adapter folder (the trained add-on)

Keep the adapter if you want to use it in week 3. Optional extensions are described in the README.

Roihu reminders

The lab reservation

GPUs are reserved for the course from roughly **16:00 to 20:00**, so jobs started in the session can finish after it.

Check your job with `squeue --me` and follow its output with `tail -f logs/lab2-lora-<jobid>.out`.

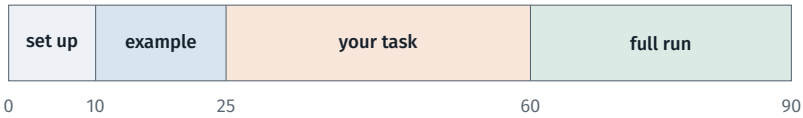
Next scheduled maintenance: **1–2 December**.

Good practice

- use `gputest` (15 minutes) for tests and `gpumedium` for the full run
- **close GPU Jupyter sessions you are not using**: a forgotten session can block a GPU for up to twelve hours
- do not run models on the login node, the machine you log in to; it has no GPU and is shared by everyone
- after a job, `seff <jobid>` shows how much time and memory it used

Install the Python packages once on the login node (README, step 3). The GPU job uses them automatically.

Leave this on the wall



- [ ] smoke test passed (step 2)
- [ ] example job finished on `gputest` (step 4)
- [ ] splits made and `check_data.py` clean (step 5)
- [ ] prompt adapted and read in a test run (steps 6–7)
- [ ] full job submitted to `gpumedium` (step 8)
- [ ] table printed with `summarize_results.py`
- [ ] ten raw generations read with `show_examples.py`
- [ ] loss plot checked, results and config saved

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.

Appendix: things that will go wrong

Score is exactly 0 or 100, or many `None`
Read the raw outputs: `show_examples.py --run <name>`. Do the labels in `TASK_SYSTEM`, the config and the data match?

`KeyError: 'text'` (or another field)
`TASK_FIELDS` does not match your JSONL fields. Run `check_data.py`.

`ModuleNotFoundError: trl` (or `peft`)
Repeat step 3: `module load`, then `pip install --user`.

`CUDA out of memory` (the GPU is full)
In the config, halve `per_device_train_batch_size` and double `gradient_accumulation_steps`.

Job waits as `PD` (pending) for a long time
`squeue --me --start` estimates the start time. `scancel <jobid>` cancels the job.

No log file
`logs/` does not exist: `mkdir -p logs` and resubmit.

Job stops at the time limit
Submit to `gpumedium` with a longer `--time`, or use smaller dev and test sets.

Loss is `nan` (not a number)
Lower the learning rate.